

Análisis Sintáctico

Análisis Sintáctico

- **Sintáxis y Semántica de un Lenguaje de Programación.**

Sintáxis

- Indica cómo se ve un programa
- Representación textual o estructura
- Es posible una representación formal precisa.
 - Gramática Libre de Contexto – Notación BNF

Semántica

- Indica cual es el significado de un programa
- Es más difícil de formalizar.

Análisis Sintáctico

- **Gramáticas**

Proveen una especificación sintáctica precisa y fácil de entender de un lenguaje de programación.

A partir de ciertas clases de gramáticas podemos construir automáticamente un analizador sintáctico eficiente.

Imparte una estructura a un lenguaje que es útil para traducir programas fuente en programas objetos correctos y detecta errores.

Facilita la evolución de un lenguaje. Se agregan e integran nuevas construcciones con facilidad.

Análisis Sintáctico

- Función del Analizador Sintáctico

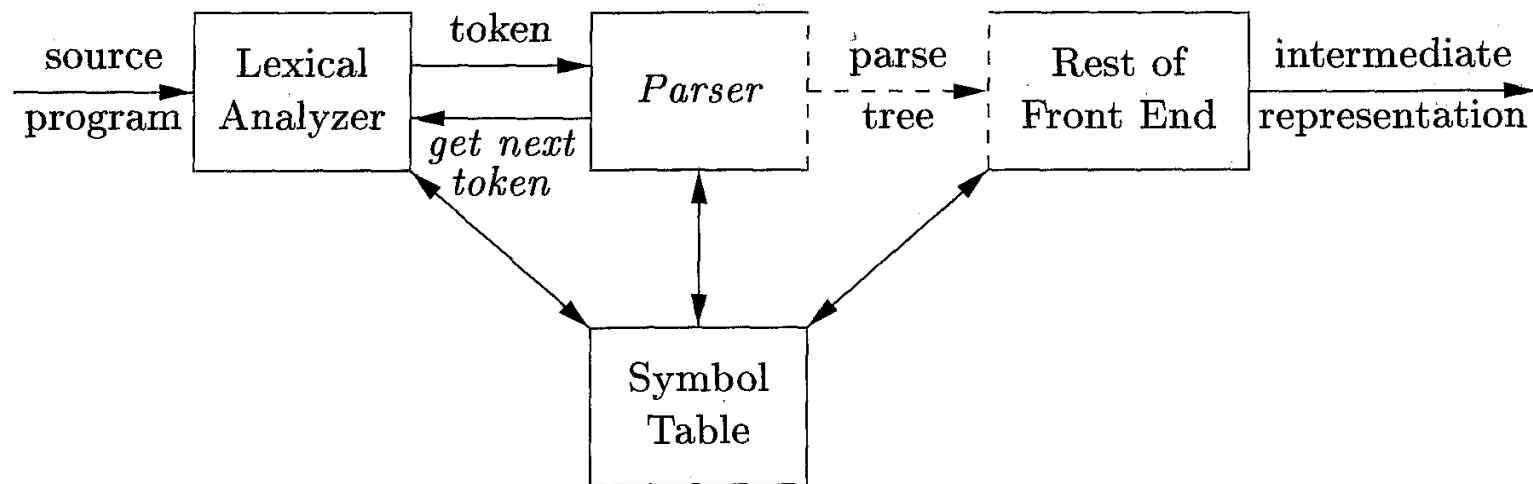


Figure 4.1: Position of parser in compiler model

Análisis Sintáctico

- **Tipos de Analizadores Sintácticos**

Universales:

pueden analizar cualquier gramática. Son ineficientes. Algoritmo de Cocke-Younger-Kasami. Algoritmo de Earley.

Descendentes:

construyen árboles de análisis sintácticos de la raíz a las hojas. Lee la entrada de izquierda a derecha, usualmente un símbolo por vez.

Ascendentes:

construyen árboles de análisis sintácticos de las hojas hacia la raíz. Lee la entrada de izquierda a derecha, usualmente un símbolo por vez.

Los métodos ascendentes y descendentes más eficientes funcionan sobre subclases de gramáticas. LL y LR describen la mayoría de las construcciones sintácticas presentes en los lenguajes de programación.

Análisis Sintáctico

- **Manejo de Errores Sintácticos**

La mayoría de los errores son detectados en esta fase.

El manejador de errores de un AS tiene por objetivo:

- Reportar la presencia de errores con claridad y precisión.
- Recuperarse de cada error rápidamente para poder seguir detectando más errores.
- No degradar la eficiencia del compilador.

Análisis Sintáctico

- **Recuperación de Errores Sintácticos**

El método más simple es que el AS termine con un mensaje de error cuando detecta el primer error.

Si puede restaurarse a un estado en el que pueda continuar el procesamiento de la entrada, debe evitar que se apilen los errores. Avalancha de errores falsos.

Estrategias:

- Recuperación en modo pánico.
- Recuperación a nivel de frase
- Producciones de error
- Corrección Global

Análisis Sintáctico

- **Recuperación de Errores Sintácticos**

Modo Pánico

- Al detectar un error el AS descarta símbolos de entrada hasta encontrar un token de sincronización (en gral. delimitadores (. ; })
- El diseñador debe seleccionar los tokens de sincronización.
- Puede ignorar una considerable cantidad de código.
- Simple de implementar
- Garantiza que no entra en un ciclo infinito.

Análisis Sintáctico

- **Recuperación de Errores Sintácticos**

A nivel de Frase

- Al detectar un error el AS puede sustituir un prefijo de la entrada restante por alguna cadena que le permita continuar. Ej: sustituir “,” por “;”, eliminar un “;”, insertar un “;”.
- Simple de implementar
- Dificultades en situaciones donde el error ha ocurrido mucho antes de donde es detectado. Ej: un begin de más.

Análisis Sintáctico

- **Recuperación de Errores Sintácticos**

Producciones de error

- Aumentar la gramática original con producciones que permiten reconocer construcciones con errores frecuentes.
- Requiere análisis de la frecuencia de errores.
- El manejo de errores queda contenido en el AS
- Permite que los mensajes de error y los procedimientos de recuperación sean específicos al error detectado.

Análisis Sintáctico

- **Recuperación de Errores Sintácticos**

Corrección Global

- Idealmente, ante un error, el AS haría el mínimo número de cambios para obtener un programa sintácticamente correcto.
- Existen algoritmos para determinar la mínima secuencia de cambios.
- Muy costoso en términos de tiempo de ejecución y espacio de almacenamiento.
- Son técnicas solo de interés teórico .

Especificación Sintáctica

- **Gramáticas Libres de Contexto**

Consisten de :

- Un conjunto finito de terminales (tokens)
- Un conjunto finito de no terminales (variables sintácticas)
- Un conjunto finito de reglas de producción de la forma
 - $A \rightarrow \alpha$ A es no terminal, α string de terminales y no terminales (incluyendo cadena nula)
- Un símbolo inicial (no terminal)

<i>expression</i>	$\rightarrow$	<i>expression</i> + <i>term</i>
<i>expression</i>	$\rightarrow$	<i>expression</i> - <i>term</i>
<i>expression</i>	$\rightarrow$	<i>term</i>
<i>term</i>	$\rightarrow$	<i>term</i> * <i>factor</i>
<i>term</i>	$\rightarrow$	<i>term</i> / <i>factor</i>
<i>term</i>	$\rightarrow$	<i>factor</i>
<i>factor</i>	$\rightarrow$	(<i>expression</i>)
<i>factor</i>	$\rightarrow$	id

Figure 4.2: Grammar for simple arithmetic expressions

Especificación Sintáctica

- **GLC. Convenciones de Notación**

Símbolos terminales:

Primeras letras minúsculas del alfabeto: a, b, c

Símbolos de operadores: $+, *$

Símbolos de puntuación: $(,), ;$

Dígitos: $0, 1, 2, \dots, 9$

Cadenas en negritas: **if, id**

Símbolos no terminales:

Letras mayúsculas del alfabeto: $A, B, C \dots E, T, F$

Símbolo inicial: por ejemplo S

Nombres en cursiva y minúsculas: *expr, instr*

Símbolos gramaticales (terminales o no terminales): X, Y, Z

Cadenas de terminales: $u, v, \dots, z$

Cadenas de símbolos gramaticales: $\alpha, \beta, \gamma \quad A \rightarrow \alpha$

Especificación Sintáctica

- **GLC. Derivaciones**

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid - E \mid (E) \mid id$$

$$E \Rightarrow E + E \Rightarrow id + E \Rightarrow id + id$$

Derivación es una secuencia de reemplazos de símbolos no terminales

En gral. un paso de derivación es:

$\alpha A \beta \Rightarrow \alpha \gamma \beta$ si existe una regla $A \rightarrow \gamma$ en nuestra gramática

donde α y β son cadenas de terminales y no-terminales

$$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n \quad (\alpha_n \text{ se deriva desde } \alpha_1 \text{ o } \alpha_1 \text{ deriva } \alpha_n)$$

$\Rightarrow$: deriva en un paso

$\Rightarrow^*$: deriva en cero o más pasos

$\Rightarrow^+$: deriva en uno o más pasos

Especificación Sintáctica

- **GLC. Terminología**

$L(G)$ es el “lenguaje de G ” (el lenguaje generado por G). Un conjunto de sentencias

Una sentencia de $L(G)$ es un string de símbolos terminales de G .

Si S es el símbolo inicial de G entonces

ω es una sentencia de $L(G)$ sssi $S \xRightarrow{*} \omega$, donde ω es un string de terminales de G .

Si G es una GLC, $L(G)$ pertenece a la clase de “lenguajes libres de contexto”.

Dos gramáticas son “equivalentes” si generan el mismo lenguaje.

$S \xRightarrow{+} \alpha$ si α contiene no-terminales, es llamada “forma sentencial” de G
si α no contiene no-terminales, es llamada “sentencia” de G .

Especificación Sintáctica

- **GLC. Ejemplo Derivación**

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$$

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(E+id) \Rightarrow -(id+id)$$

Si en cada paso de derivación elegimos el no-terminal de más a la izquierda, es una “derivación izquierda” (left-most derivation).

Si en cada paso de derivación elegimos el no-terminal de más a la derecha, es una “derivación derecha” (right-most derivation).

Los AS descendentes (top-down) tratan de encontrar la derivación izquierda del programa fuente

Los AS ascendentes (bottom-up) tratan de encontrar la derivación derecha invertida o vista en reversa del programa fuente

Especificación Sintáctica

- GLC. Árboles de Derivación**

Representación gráfica de una derivación

Los nodos internos son símbolos no-terminales

Las hojas son símbolos terminales

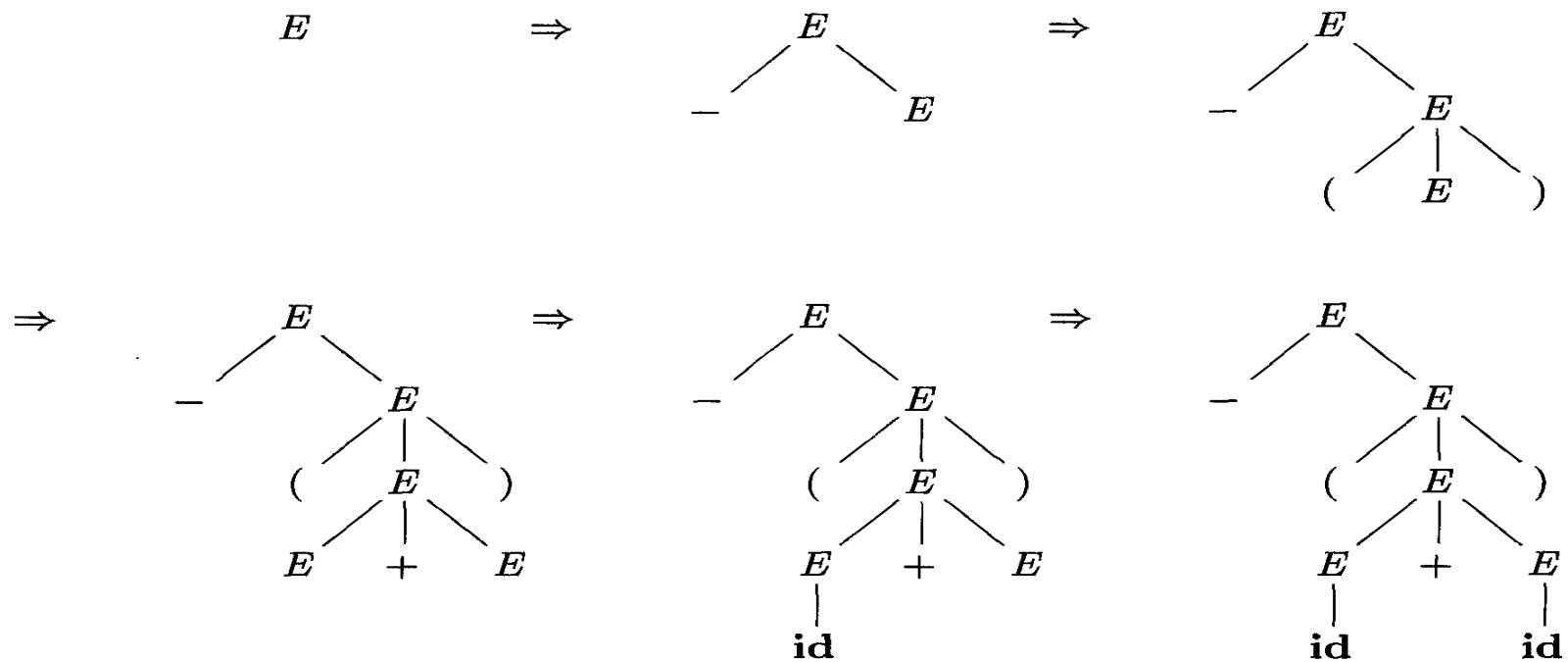


Figure 4.4: Sequence of parse trees for derivation (4.8)

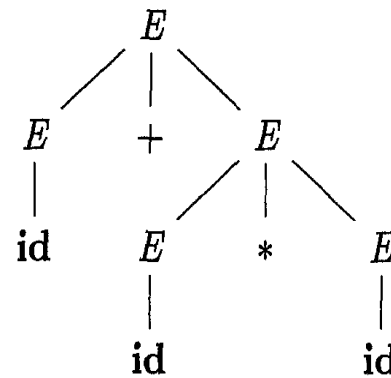
Especificación Sintáctica

- GLC. Ambigüedad**

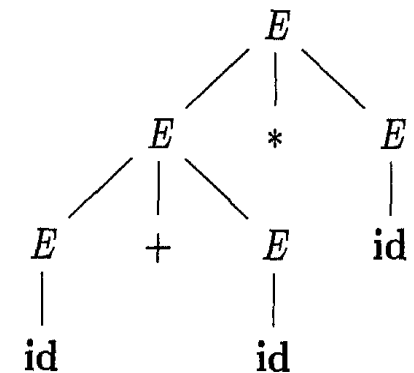
Una gramática que produce más de un árbol de derivación (distinto) para una sentencia, es denominada “ambigua”.

Una gramática ambigua es aquella que produce más de una derivación por la izquierda, o más de una derivación por la derecha para la misma sentencia.

$E \Rightarrow E + E$	$E \Rightarrow E * E$
$\Rightarrow id + E$	$\Rightarrow E + E * E$
$\Rightarrow id + E * E$	$\Rightarrow id + E * E$
$\Rightarrow id + id * E$	$\Rightarrow id + id * E$
$\Rightarrow id + id * id$	$\Rightarrow id + id * id$



(a)



(b)

Figure 4.5: Two parse trees for **id+id*id**

Especificación Sintáctica

- **GLC. Ambigüedad**

Para la mayoría de los Analizadores Sintácticos, la gramática debe ser “no ambigua”.

Deberíamos eliminar la ambigüedad de la gramática durante la etapa de diseño del compilador. A veces no es posibles (lenguajes inherentemente ambiguos)

En algunos casos podemos “manejar la ambigüedad”. Debemos poder elegir uno de los posibles árboles de derivación para una sentencia.

Especificación Sintáctica

- GLC. Ambigüedad. Ejemplo Clásico

$stmt \rightarrow$ $if\ expr\ then\ stmt$
 $if\ expr\ then\ stmt\ else\ stmt$
 $other$

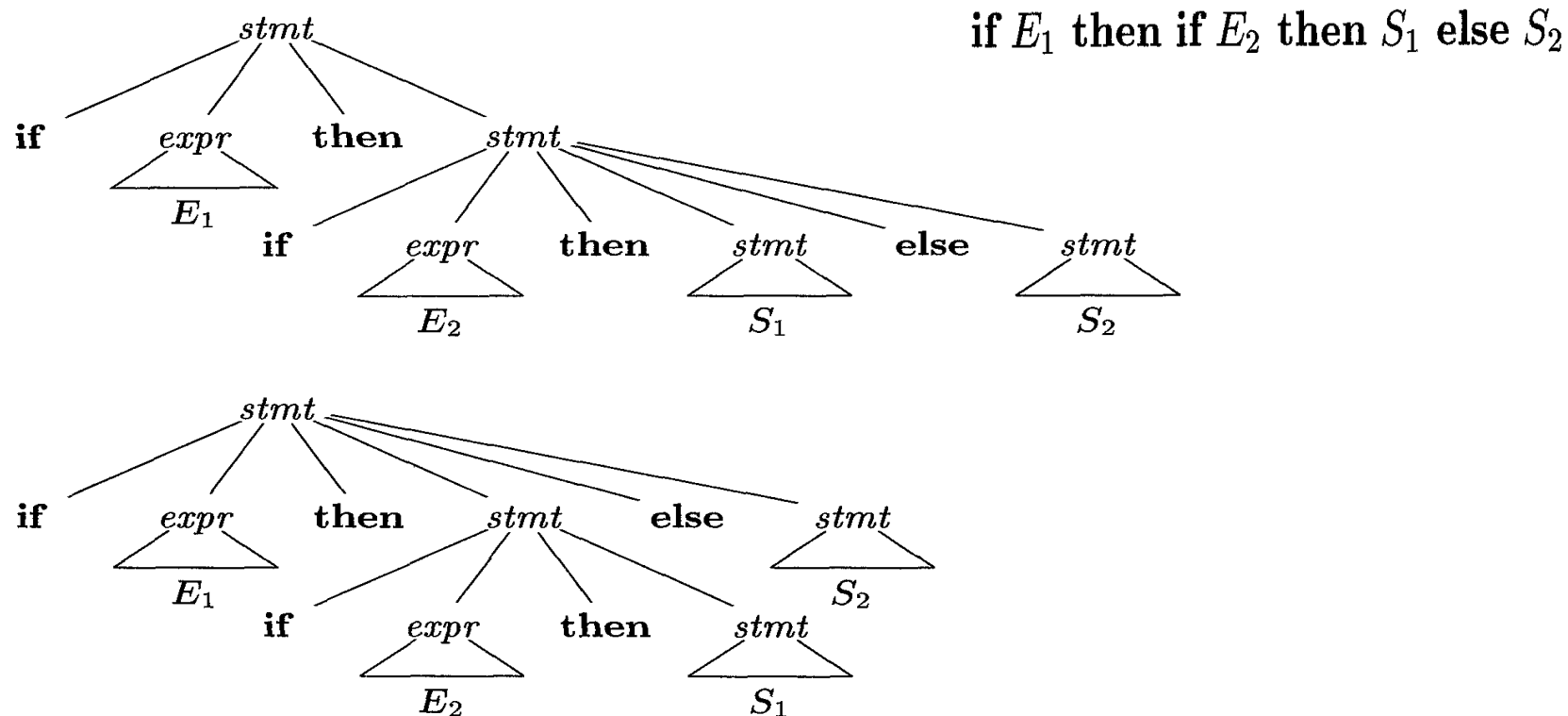


Figure 4.9: Two parse trees for an ambiguous sentence

Especificación Sintáctica

- GLC. Ambigüedad. Ejemplo Clásico

<i>stmt</i>	→	<i>matched_stmt</i>
		<i>open_stmt</i>
<i>matched_stmt</i>	→	if <i>expr</i> then <i>matched_stmt</i> else <i>matched_stmt</i>
		other
<i>open_stmt</i>	→	if <i>expr</i> then <i>stmt</i>
		if <i>expr</i> then <i>matched_stmt</i> else <i>open_stmt</i>

Figure 4.10: Unambiguous grammar for if-then-else statements

Especificación Sintáctica

- **GLC. Ambigüedad y Precedencia de Operadores**

En algunos casos, la ambigüedad de una gramática puede resolverse usando información de precedencia y reglas de asociatividad.

$$E \rightarrow E+E \mid E^*E \mid E^{\wedge}E \mid \text{id} \mid (E)$$

precedencia: $\wedge$ (der a izq)
 $*$ (izq a der)
 $+$ (izq a der)

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T^*F \mid F$$

$$F \rightarrow G^{\wedge}F \mid G$$

$$G \rightarrow \text{id} \mid (E)$$

Especificación Sintáctica

- **GLC. Recursión a Izquierda**

Una gramática es “recursiva a izquierda” si tiene un no-terminal A tal que existe una derivación de la forma:

$$A \xRightarrow{+} A\alpha \quad \text{para algún string } \alpha$$

Las técnicas de AS descendentes no pueden manejar la recursión a izquierda.

Debemos convertir la gramática recursiva a izquierda en una gramática equivalente sin recursión a izquierda.

La recursión a izquierda puede aparecer en un solo paso de derivación (recursión a izquierda inmediata) o en más de un paso.

Especificación Sintáctica

- **GLC. Recursión a Izquierda**

Una gramática es “recursiva a izquierda” si tiene un no-terminal A tal que existe una derivación de la forma:

$$A \xRightarrow{+} A\alpha \quad \text{para algún string } \alpha$$

Las técnicas de AS descendentes no pueden manejar la recursión a izquierda.

Debemos convertir la gramática recursiva a izquierda en una gramática equivalente sin recursión a izquierda.

La recursión a izquierda puede aparecer en un solo paso de derivación (recursión a izquierda inmediata) o en más de un paso.

Especificación Sintáctica

- **GLC. Eliminando Recursión a Izquierda Inmediata**

$$A \rightarrow A \alpha \mid \beta \quad \text{donde } \beta \text{ no comienza con } A$$

$\Downarrow$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \lambda \quad A' \text{ es un nuevo no-terminal}$$

En general:

$$A \rightarrow A \alpha_1 \mid \dots \mid A \alpha_m \mid \beta_1 \mid \dots \mid \beta_n \quad \text{donde } \beta_1 \dots \beta_n \text{ no comienzan con } A$$

$\Downarrow$

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \lambda$$

Especificación Sintáctica

- **GLC. Eliminando Recursión a Izquierda Inmediata**

Ejemplo

$$E \rightarrow E+T \mid T$$
$$T \rightarrow T*F \mid F$$
$$F \rightarrow \text{id} \mid (E)$$

$$E \rightarrow T E'$$
$$E' \rightarrow +T E' \mid \lambda$$
$$T \rightarrow F T'$$
$$T' \rightarrow *F T' \mid \lambda$$
$$F \rightarrow \text{id} \mid (E)$$

Especificación Sintáctica

- **GLC. Recursión a Izquierda. Problema**

Una gramática puede no ser recursiva a izquierda de manera inmediata, pero aún ser recursiva a izquierda.

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Sc \mid d$$

Aquí, no hay recursión a izquierda inmediata.

Pero..

$$\underline{S} \Rightarrow Aa \Rightarrow \underline{S}ca \quad \text{o}$$

$$\underline{A} \Rightarrow Sc \Rightarrow \underline{A}ac \quad \text{causa recursión a izquierda}$$

Debemos eliminar todas las formas de recursión a izquierda.

Especificación Sintáctica

- **GLC. Recursión a Izquierda. Algoritmo de Eliminación.**

Ejemplo

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid f \end{aligned}$$

Ordenamos los no-terminales: S, A (por ejemplo)

Para S: no entramos en el loop interno, no hay recursión a izquierda inmediata para S.

Para A: reemplazamos $A \rightarrow Sd$ con $A \rightarrow Aad \mid bd$

Luego, tendríamos $A \rightarrow Ac \mid Aad \mid bd \mid f$

Eliminamos la recursión a izquierda inmediata para A:

$$\begin{aligned} A &\rightarrow bdA' \mid fA' \\ A' &\rightarrow cA' \mid adA' \mid \lambda \end{aligned}$$

Finalmente, la gramática equivalente sin recursión a izquierda es:

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow bdA' \mid fA' \\ A' &\rightarrow cA' \mid adA' \mid \lambda \end{aligned}$$

Especificación Sintáctica

- **Factorización a Izquierda**

El análisis predictivo (descendente sin backtracking) requiere que la gramática esté “factorizada a izquierda”.

La factorización por izquierda es una transformación gramatical.

Ejemplo:

$$\begin{array}{lcl} stmt & \rightarrow & \text{if } expr \text{ then } stmt \text{ else } stmt \\ & | & \text{if } expr \text{ then } stmt \end{array}$$

Cuando vemos if, no podemos determinar que regla de producción seguir

Especificación Sintáctica

- **Factorización a Izquierda. Algoritmo**

Para cada no-terminal A con dos o más alternativas (reglas de producción) con un prefijo común no vacío α ,

$$A \rightarrow \alpha\beta_1 \mid \dots \mid \alpha\beta_n \mid \gamma_1 \mid \dots \mid \gamma_m$$

debemos convertirla en :

$$A \rightarrow \alpha A' \mid \gamma_1 \mid \dots \mid \gamma_m$$

$$A' \rightarrow \beta_1 \mid \dots \mid \beta_n$$

Especificación Sintáctica

- Factorización a Izquierda. Ejemplo

$$A \rightarrow \underline{a}bB \mid \underline{a}B \mid cdg \mid cdeB \mid cdfB$$
$$\Downarrow$$
$$A \rightarrow aA' \mid \underline{cd}g \mid \underline{cde}B \mid \underline{cdf}B$$
$$A' \rightarrow bB \mid B$$
$$\square$$
$$A \rightarrow aA' \mid cdA''$$
$$A' \rightarrow bB \mid B$$
$$A'' \rightarrow g \mid eB \mid fB$$

Especificación Sintáctica

- Factorización a Izquierda. Ejemplo

$$A \rightarrow ad \mid a \mid ab \mid abc \mid b$$

□

$$A \rightarrow aA' \mid b$$

$$A' \rightarrow d \mid \lambda \mid b \mid bc$$

□

$$A \rightarrow aA' \mid b$$

$$A' \rightarrow d \mid \lambda \mid bA''$$

$$A'' \rightarrow \lambda \mid c$$

Introducción al Análisis Sintáctico

- El Análisis Sintáctico (o Parsing) es el proceso de determinar si una cadena de tokens puede ser generada por una gramática.
- Se trata de construir un árbol de derivación, aún cuando no se lo construya explícitamente. El Analizador Sintáctico (o Parser) debe ser capaz de generarlo.
- Para cualquier GLC existe un parser que requiere de un tiempo de ejecución de $O(n^3)$ (como máximo) para analizar una cadena de n tokens.
- Para cierta clase de GLC existen parser más eficientes

Análisis Sintáctico

- **Métodos principales de Análisis Sintáctico**
 - Top-Down o Descendentes
 - La construcción del árbol de derivación comienza desde la raíz y procede hacia las hojas.
 - Pueden construirse parsers eficientes sin herramientas.
 - Intentan encontrar una “derivación izquierda” (leftmost derivation)
 - Bottom-Up o Ascendentes
 - La construcción del árbol de derivación se realiza desde las hojas hacia la raíz.
 - Manejan una clase más extensa de gramáticas. Las herramientas de software para generar a.s. lo usan con frecuencia .
 - Intentan encontrar una “derivación derecha” (rightmost derivation)

Análisis Sintáctico

- **Top-Down Parsing o Descendentes**

El proceso de construcción top-down del árbol de análisis sintáctico comienza con la raíz, etiquetada con el símbolo inicial de la gramática y repitiendo los siguientes pasos :

- En el nodo n , etiquetado con el no terminal A , seleccionar una de las producciones para A y crear los hijos de n con los símbolos del lado derecho de la regla.
- Encontrar el siguiente nodo n' para construir un subarbol con raíz n' . Usualmente, este nodo es el rotulado con el no terminal sin expandir de más a la izquierda del árbol actual.

Análisis Sintáctico

- **Top-Down Parsing** o **Descendentes**

Consideremos la siguiente gramática :

$$\begin{aligned} instr &\rightarrow \mathbf{expr} ; \\ &\quad | \mathbf{if} (\mathbf{expr}) instr \\ &\quad | \mathbf{for} (\mathit{expropc} ; \mathit{expropc} ; \mathit{expropc}) instr \\ &\quad | \mathbf{otras} \end{aligned}$$
$$\mathit{expropc} \rightarrow \lambda \mid \mathbf{expr}$$

Veamos el análisis sintáctico de la cadena :

for (; expr ; expr) otras

Al token actual de la entrada se lo denomina “símbolo de preanálisis” (símbolo de lookahead).

La selección para un no-terminal puede involucrar prueba y error. Esto puede implicar retroceso o backtracking.

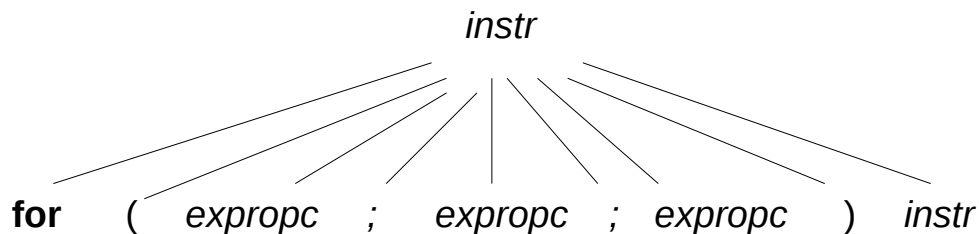
Análisis Sintáctico

- **Top-Down Parsing** o **Descendentes**

Veamos el análisis sintáctico de la cadena :

for (; expr ; expr) otras

- Inicialmente, el símbolo de preanálisis es el token **for**. La raíz del á.s. corresponde al símbolo inicial *instr*.
- Encontramos la única producción que comienza con **for** y construimos los nodos correspondientes al lado derecho de la producción.



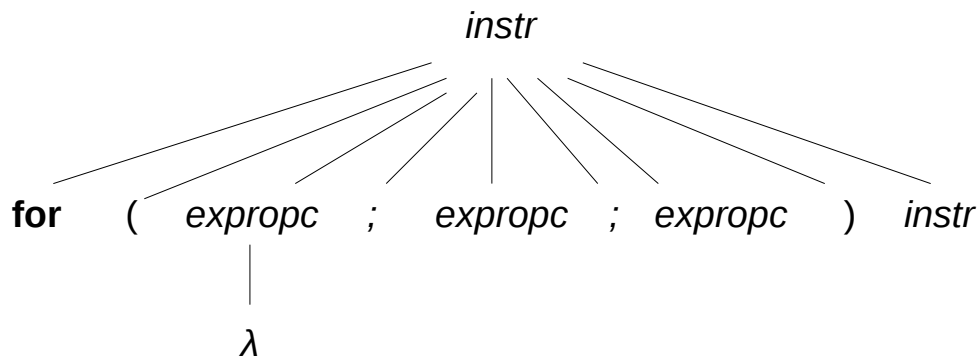
Análisis Sintáctico

- **Top-Down Parsing o Descendentes**

Seleccionamos el hijo extremo izquierdo como el nodo actual. Como es un terminal y coincide con el símbolo de preanálisis, avanzamos tanto en el árbol como en la cadena de entrada.

El siguiente nodo es “(“, idem anterior, avanzamos en el árbol y en la entrada.

Alcanzamos el nodo para *expropc* y se intenta con cada producción hasta alcanzar un match con el símbolo de preanálisis. Entonces aplicamos la primer producción para *expreopc*

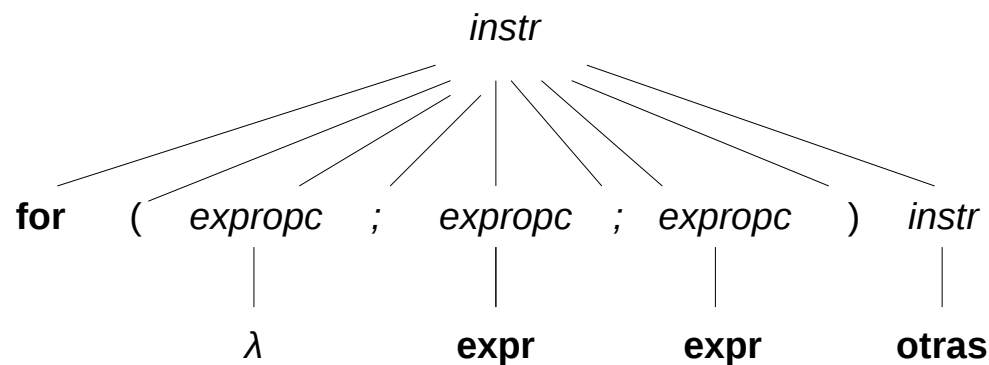


Análisis Sintáctico

- **Top-Down Parsing** o **Descendentes**

Actuamos de la misma manera con el resto de los nodos

El proceso termina cuando todos los nodos hojas son terminales



Análisis Sintáctico

- **Análisis Sintáctico Descendente.**
 - Análisis Sintáctico Predictivo Recursivo
 - Análisis Sintáctico Predictivo No Recursivo

Análisis Sintáctico

- **Análisis Descendente Recursivo**

Es un método top-down de análisis en el que se ejecutan un conjunto de procedimientos recursivos para procesar la entrada.

Se asocia un procedimiento para cada no terminal de la gramática.

La secuencia de procedimientos llamadas para procesar la entrada define implícitamente el árbol de derivación.

- **Análisis Descendente Recursivo Predictivo**

Es una forma especial de análisis descendente recursivo donde el símbolo de preanálisis (lookahead) determina unívocamente (sin ambigüedad) el procedimiento seleccionado para cada no terminal.

Análisis Sintáctico

- Analizador Sintáctico Predictivo para la gramática :

```
instr → expr ;  
      | if ( expr ) instr  
      | for ( expropc ; expropc ; expropc ) instr  
      | otras  
expropc →  $\lambda$  | expr
```

- Procedimientos para los no terminales *instr* y *expropc*
- Procedimiento adicional *match(t)*, compara el argumento *t* con el símbolo de preanálisis y avanza al siguiente terminal de entrada si coinciden.

```
void match(terminal t) {  
    if ( preanálisis == t ) preanálisis = siguienteTerminal;  
    else reportar("error de sintaxis");  
}
```

Análisis Sintáctico

- Analizador Sintáctico Predictivo para la gramática :

```
void instr() {
    switch ( preanálisis ) {
        case expr :
            match(expr); match(';'); break;
        case if :
            match(if); match('('); match(expr); match(')'); instr(); break;
        case for :
            match(for); match('(');
            expropc(); match(';'); expropc(); match(';'); expropc();
            match(')'); instr(); break;
        case otras :
            match(otras); break
        default:
            reportar("error de sintaxis");
    }
}

void expropc() {
    if ( preanálisis == expr ) match(expr);
}
```

Análisis Sintáctico

- **Uso del Analizador Sintáctico Predictivo**

Para comenzar el análisis llamamos al procedimiento *instr* asociado al símbolo inicial *instr*. Inicialmente, el símbolo de preanálisis es “for”, dado que es el primer terminal (token) de la entrada

Cuando el símbolo de preanálisis coincide con el nodo actual del árbol, el procedimiento `match()` devuelve el siguiente token de la entrada.

El análisis predictivo depende de la información de los primeros símbolos que aparecen en el lado derecho de una producción.

Análisis Descendente Predictivo

- **El Conjunto Primero**

Formalmente, el analizador predictivo usa el conjunto Primero o First.

- Sea α una cadena de símbolos gramaticales (terminales y/o no terminales).
 - Definimos $\text{Primero}(\alpha)$ como el conjunto de terminales (tokens) que aparecen como los primeros símbolos de una o más cadenas generadas por α .
- Si $\alpha = \lambda$ o puede generar λ , entonces λ también pertenece a $\text{Primero}(\alpha)$.

Ejemplos :

$\text{PRIMERO}(\text{instr}) = \{\mathbf{expr}, \mathbf{if}, \mathbf{for}, \mathbf{otras}\}$

$\text{PRIMERO}(\mathbf{expr} ;) = \{\mathbf{expr}\}$

Análisis Descendente Predictivo

- **El Conjunto Primero**

Si existen dos producciones tal que $A \rightarrow \alpha$ y $A \rightarrow \beta$, entonces el analizador predictivo consulta los conjuntos PRIMERO de α y β para decidir que hacer.

- Si el símbolo de preanálisis está en $\text{PRIMERO}(\alpha)$, entonces α es usado.
- Si el símbolo de preanálisis está en $\text{PRIMERO}(\beta)$, entonces β es usado.

El análisis descendente recursivo predictivo (sin backtracking) requiere que

$$\text{PRIMERO}(\alpha) \cap \text{PRIMERO}(\beta) = \Phi \text{ (vacío)}$$

Análisis Descendente Predictivo

- **Caso Especial : las λ -Producciones**

El analizador descendente usará una λ -producción (existente) por defecto cuando ninguna otra producción puede ser usada.

En nuestro ejemplo (modificado con *expr*)

```
instr → expr ;  
      | if ( expr) instr  
      | for ( expropc ; expropc ; expropc ) instr  
      | otras
```

```
expropc →  $\lambda$  | expr
```

Durante el análisis de *expropc*, si el símbolo de preanálisis no está en PRIMERO(*expr*), entonces se usa la λ -producción (el procedimiento asociado termina).

Análisis Descendente Predictivo

- **Diseño de un Analizador Sintáctico Predictivo**

El analizador sintáctico predictivo consiste en un procedimiento para cada no-terminal de una gramática. Cada procedimiento realiza lo siguiente :

- Decide que producción usar en base al símbolo de preanálisis. Si el símbolo de preanálisis está en $\text{PRIMERO}(\alpha)$ para alguna producción $A \rightarrow \alpha$, entonces esta producción es usada. Si existe un conflicto entre los lados derechos, el método falla. Si el símbolo de preanálisis no pertenece a ningún conjunto PRIMERO , entonces la λ -producción (existente) es usada.
- El procedimiento “usa” una producción, o la aplica, “imitando” el lado derecho en términos de código, llamando a la función `match()` y a los restantes procedimientos.

Análisis Descendente Predictivo

- **Recursividad por izquierda**

El analizador sintáctico predictivo recursivo puede ciclar indefinidamente con gramáticas recursivas por izquierda.

Por ejemplo :

$$expr \rightarrow exp + term$$

La recursión a izquierda puede ser eliminada.

Sea el no terminal A tal que :

$$A \rightarrow A\alpha \mid \beta$$

Entonces, agregamos un nuevo no terminal R y modificamos las reglas para A :

$$\begin{aligned} A &\rightarrow \beta R \\ R &\rightarrow \alpha R \mid \lambda \end{aligned}$$

Análisis Descendente Predictivo

- **Consideraciones Finales**

El analizador sintáctico predictivo usa un solo símbolo de preanálisis para decidir que producción aplicar.

Es un analizador LL(1) dado que lee la entrada de izquierda (Left) a derecha e intenta encontrar una derivación izquierda (Leftmost).

El algoritmo de análisis sintáctico presentado solo se aplica a un limitado grupo de gramáticas, conocidas como gramáticas LL(1).

Este grupo es el tipo más común de gramáticas, pero existen otras clases de gramáticas que las contienen y para las cuales también construiremos analizadores sintácticos.

Análisis Sintáctico Predictivo No Recursivo

El analizador usa un buffer de entrada, una pila, una tabla de análisis sintáctico y genera una salida

El final del buffer de entrada y el fondo de la pila están delimitados con el signo \$.

La pila contiene tanto símbolos terminales como no-terminales. Inicialmente la pila contiene solo al símbolo inicial de la gramática encima de "\$". Es decir, la pila contiene "S\$" y la cadena de entrada es "w\$".

La tabla de análisis es un arreglo bidimensional $M[A,a]$, donde A es un no-terminal y a es un terminal o \$.

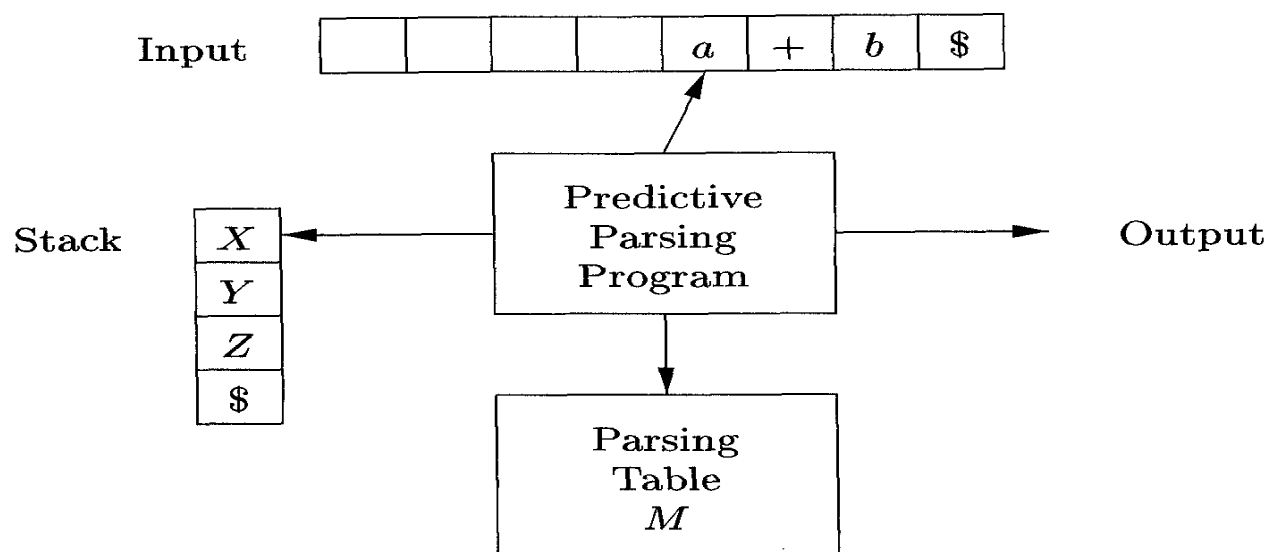


Figure 4.19: Model of a table-driven predictive parser

Análisis Sintáctico Predictivo No Recursivo

- **Funcionamiento**

El analizador tiene en cuenta a X (el símbolo en el tope de la pila) y a (el símbolo actual de la entrada). Estos símbolos determinan la acción del AS:

Si $X=a=\$$, el análisis finaliza con éxito.

Si a es terminal y $X=a$, el AS desapila X y avanza al siguiente símbolo de la entrada.

Si X es un no-terminal, se consulta $M[X,a]$:

Si $M[X,a]=\{X \rightarrow UVW\}$, el AS reemplaza X en el tope de la pila por WVU (U en el tope). La salida es la producción usada, $X \rightarrow UVW$.

Si $M[X,a]=\emptyset$, el análisis falla y la cadena de entrada no pertenece al lenguaje generado por la gramática. La salida es "error". Posiblemente se llame a una rutina de recuperación de errores.

Análisis Sintáctico Predictivo No Recursivo

- Ejemplo

G' gramática de expresiones, luego de eliminar la recursión a izquierda.

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \lambda$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \lambda$$

$$F \rightarrow \text{id} \mid (E)$$

	id	+	*	(	)	\$
<i>E</i>	$E \rightarrow T E'$			$E \rightarrow T E'$		
<i>E'</i>		$E' \rightarrow + T E'$			$E' \rightarrow \lambda$	$E' \rightarrow \lambda$
<i>T</i>	$T \rightarrow F T'$			$T \rightarrow F T'$		
<i>T'</i>		$T' \rightarrow \lambda$	$T' \rightarrow * F T'$		$T' \rightarrow \lambda$	$T' \rightarrow \lambda$
<i>F</i>	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

MATCHED	STACK	INPUT	ACTION
	<i>E</i> \$	id + id * id\$	
	<i>TE'</i> \$	id + id * id\$	output $E \rightarrow T E'$
	<i>FT'E'</i> \$	id + id * id\$	output $T \rightarrow F T'$
	id <i>T'E'</i> \$	id + id * id\$	output $F \rightarrow \text{id}$
id	<i>T'E'</i> \$	+ id * id\$	match id
id	<i>E'</i> \$	+ id * id\$	output $T' \rightarrow \epsilon$
id	+ <i>TE'</i> \$	+ id * id\$	output $E' \rightarrow + T E'$
id +	<i>TE'</i> \$	id * id\$	match +
id +	<i>FT'E'</i> \$	id * id\$	output $T \rightarrow F T'$
id +	id <i>T'E'</i> \$	id * id\$	output $F \rightarrow \text{id}$
id + id	<i>T'E'</i> \$	* id\$	match id
id + id	* <i>FT'E'</i> \$	* id\$	output $T' \rightarrow * F T'$
id + id *	<i>FT'E'</i> \$	id\$	match *
id + id *	id <i>T'E'</i> \$	id\$	output $F \rightarrow \text{id}$
id + id * id	<i>T'E'</i> \$	\$	match id
id + id * id	<i>E'</i> \$	\$	output $T' \rightarrow \epsilon$
id + id * id	\$	\$	output $E' \rightarrow \epsilon$

Figure 4.21: Moves made by a predictive parser on input **id + id * id**

Análisis Sintáctico Predictivo No Recursivo

- **Construcción de la tabla de análisis M**

Función $\text{Primeros}(X)$, para todo símbolo (terminal o no-terminal) de la gramática

Si X es un terminal, entonces $\text{Primeros}(X) = \{X\}$

Si $X \rightarrow \lambda$ es una producción, entonces añadir λ a $\text{Primeros}(X)$

Si X es un no-terminal y $X \rightarrow Y_1 Y_2 \dots Y_k$, entonces añadir a en $\text{Primeros}(X)$ si a está en $\text{Primeros}(Y_i)$ y λ está en $\text{Primeros}(Y_1), \dots, \text{Primeros}(Y_{i-1})$

Para calcular Primeros de una cadena, $\text{Primeros}(X_1 X_2 \dots X_n)$:

Incluimos el valor de $\text{Primeros}(X_1) - \{\lambda\}$

si λ pertenece a $\text{Primeros}(X_1)$, incluimos a $\text{Primeros}(X_2) - \{\lambda\}$

si λ pertenece a $\text{Primeros}(X_2)$, incluimos a $\text{Primeros}(X_3) - \{\lambda\}$

...

$E \rightarrow T E'$

$E' \rightarrow +T E' \mid \lambda$

$T \rightarrow F T'$

$T' \rightarrow *F T' \mid \lambda$

$F \rightarrow \text{id} \mid (E)$

$\text{Primeros}(E) = \{ (, \text{id} \}$

$\text{Primeros}(E') = \{ +, \lambda \}$

$\text{Primeros}(T) = \{ (, \text{id} \}$

$\text{Primeros}(T') = \{ *, \lambda \}$

$\text{Primeros}(F) = \{ (, \text{id} \}$

Análisis Sintáctico Predictivo No Recursivo

- **Construcción de la tabla de análisis M**

Función $\text{Siguietes}(N)$, para cada no-terminal N

Incluir $\$$ en $\text{Siguietes}(S)$ donde S es el símbolo inicial y $\$$ el símbolo de fin de cadena

Si existe una producción de la forma $A \rightarrow \alpha B \beta$ entonces incluir $\text{Primeros}(\beta)$, excepto λ , en $\text{Siguietes}(B)$

Si existe una producción de la forma $A \rightarrow \alpha B$ o $A \rightarrow \alpha B \beta$, donde $\text{Primeros}(\beta)$ contiene a λ , todo lo que esté en $\text{Siguietes}(A)$ se incluye en $\text{Siguietes}(B)$

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \lambda$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \lambda$

$F \rightarrow \text{id} \mid (E)$

$\text{Primeros}(E) = \{ (, \text{id} \}$

$\text{Primeros}(E') = \{ +, \lambda \}$

$\text{Primeros}(T) = \{ (, \text{id} \}$

$\text{Primeros}(T') = \{ *, \lambda \}$

$\text{Primeros}(F) = \{ (, \text{id} \}$

$\text{Siguietes}(E) = \{ \$,) \}$

$\text{Siguietes}(E') = \{ \$,) \}$

$\text{Siguietes}(T) = \{ +, \$,) \}$

$\text{Siguietes}(T') = \{ +, \$,) \}$

$\text{Siguietes}(F) = \{ *, +, \$,) \}$

Análisis Sintáctico Predictivo No Recursivo

- **Construcción de la tabla de análisis M**

Entrada: Una gramática G

Salida: La tabla de Análisis Sintáctico M

Método: Para cada producción $A \rightarrow \alpha$ seguir los dos pasos siguientes:

- para cada terminal a de $\text{Primeros}(\alpha)$ añadir $A \rightarrow \alpha$ a $M[A,a]$
 - si λ está en $\text{Primeros}(\alpha)$ añadir $A \rightarrow \alpha$ a $M[A,b]$ para cada terminal b de $\text{Siguietes}(A)$
 - si λ está en $\text{Primeros}(\alpha)$ y $\$$ está en $\text{Siguietes}(A)$ añadir $A \rightarrow \alpha$ a $M[A,\$]$
- (cada entrada no definida de M es una entrada de “error”)

Análisis Sintáctico Predictivo No Recursivo

- Ejemplo

$$E \rightarrow T E'$$

$$E' \rightarrow +T E' \mid \lambda$$

$$T \rightarrow F T'$$

$$T' \rightarrow *F T' \mid \lambda$$

$$F \rightarrow \text{id} \mid (E)$$

$$\text{Primeros}(E) = \{ (, id \}$$

$$\text{Primeros}(E') = \{ +, \lambda \}$$

$$\text{Primeros}(T) = \{ (, id \}$$

$$\text{Primeros}(T') = \{ *, \lambda \}$$

$$\text{Primeros}(F) = \{ (, id \}$$

$$\text{Siguietes}(E) = \{ \$,) \}$$

$$\text{Siguietes}(E') = \{ \$,) \}$$

$$\text{Siguietes}(T) = \{ +, \$,) \}$$

$$\text{Siguietes}(T') = \{ +, \$,) \}$$

$$\text{Siguietes}(F) = \{ *, +, \$,) \}$$

Análisis Sintáctico Predictivo No Recursivo

- **Gramáticas LL(1)**

Una gramática cuya tabla de análisis no muestra entradas múltiples es llamada gramática LL(1).

Las gramáticas LL(1) tienen propiedades interesantes:

- No ambiguas
- No recursivas a izquierda
- Factorizadas a izquierda

Análisis Sintáctico Predictivo No Recursivo

- **Gramáticas LL(1). Definición**

Una gramática G es LL(1) “si y solo si” en todos los casos donde $A \rightarrow \alpha \mid \beta$ son dos producciones distintas de G se verifican las siguientes condiciones:

- Para ningún terminal a , α y β derivan cadenas que comienzan con a .
- Como máximo, solo una de las opciones α y β pueden derivar la cadena nula.
- Si β deriva la cadena nula, entonces α no deriva ninguna cadena que comience con un no terminal en $\text{Siguietes}(A)$.

Análisis Sintáctico Predictivo No Recursivo

- Ejemplo Gramática que no es LL(1).

prop	→	if exp then prop
		if exp then prop else prop
		a
		b
exp	→	p
		q

Factorizando a izquierda

prop	→	if exp then prop prop'
		a
		b
prop'	→	else prop
		λ
exp	→	p
		q

Análisis Sintáctico Predictivo No Recursivo

- Ejemplo Gramática que no es LL(1).

prop	→	if exp then prop prop'
		a
		b
prop'	→	else prop
		λ
exp	→	p
		q

Tabla de análisis obtenida:

	if	then	else	a	b	p	q	\$
<i>p</i>	$p \rightarrow \text{if exp then } p p'$			$p \rightarrow a$	$p \rightarrow b$			
<i>p'</i>			$p' \rightarrow \text{else } p$ $p' \rightarrow \lambda$					$p' \rightarrow \lambda$
<i>exp</i>						$exp \rightarrow p$	$exp \rightarrow q$	

En $M[p', \text{else}]$ tenemos dos producciones. Si elegimos siempre la regla $p' \rightarrow \text{else } p$, lo que estamos haciendo es elegir el árbol de derivación que asocia el *else* con el *then* más próximo.

Análisis Sintáctico Predictivo

- **Recuperación de errores.**

En el contexto del análisis sintáctico predictivo, los errores pueden darse por dos situaciones:

- Cuando el terminal en el tope de la pila no concuerda con el siguiente terminal de entrada.
- Cuando se tiene un no-terminal A en el tope de la pila y un símbolo a en la entrada, y el contenido de $M[A,a]$ es vacío.

Análisis Sintáctico Predictivo

- **Recuperación de errores. Modo Pánico**

Al detectar un error, se avanza en la entrada hasta alcanzar algún token de sincronización.

La eficiencia de la estrategia depende de una adecuada elección del conjunto de sincronización para cada no-terminal.

Los conjuntos deben elegirse de modo que el AS se recupere con rapidez de los errores que tengan una buena probabilidad de ocurrir en la práctica.

Análisis Sintáctico Predictivo

- **Recuperación de errores. Modo Pánico.**

Heurísticas para la elección de los tokens de sincronización:

- Para cada no-terminal A, los tokens de sincronización podrían ser aquellos pertenecientes a $\text{Siguietes}(A)$.

Si omitimos tokens hasta que se vea un elemento de $\text{Siguietes}(A)$ y sacamos A de la pila, es probable que el AS pueda continuar.

- $\text{Siguietes}(A)$ puede resultar insuficiente como conjunto de sincronización.

Ej: si un lenguaje termina sus sentencias con “,” y el error fue omitir ese carácter, a continuación seguramente encontraremos una palabra clave que inicia una sentencia.

Esta palabra clave será ignorada por no pertenecer a $\text{Siguietes}(A)$.

Podríamos incluir las palabras claves en el conjunto de sincronización para A.

Si el lenguaje está formado por construcciones organizadas de una forma jerárquica (bloques, sentencias, expresiones), podemos incluir en los conjuntos de sincronización de no-terminales inferiores de la jerarquía, a los terminales que inician las construcciones superiores. Ej : agregar palabras claves que empiezan las sentencias a los conjuntos de sincronización de los no-terminales que generan las expresiones.

Análisis Sintáctico Predictivo

- **Recuperación de errores. Modo Pánico.**

Heurísticas para la elección de los tokens de sincronización:

- Incorporar en el conjunto de sincronización para A, el contenido de $\text{Primeros}(A)$

Se continuaría el análisis al encontrar el token de sincronización, sin desapilar A.

- Si un terminal en el tope de la pila no se puede matchear, extraerlo de la pila, emitir un mensaje que indique que se insertó un terminal en la entrada y continuar el análisis.

Esto equivale a considerar como componentes de sincronización al resto de los tokens.

Análisis Sintáctico Predictivo

- Recuperación de errores. Modo Pánico.

Ejemplo

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	synch	synch
E'		$E \rightarrow +TE'$			$E \rightarrow \epsilon$	$E \rightarrow \epsilon$
T	$T \rightarrow FT'$	synch		$T \rightarrow FT'$	synch	synch
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$	synch	synch	$F \rightarrow (E)$	synch	synch

Figure 4.22: Synchronizing tokens added to the parsing table of Fig. 4.17

On the erroneous input $) \text{id} * + \text{id}$, the parser and error recovery mechanism of Fig. 4.22 behave as in Fig. 4.23. $\square$

STACK	INPUT	REMARK
$E \$$	$) \text{id} * + \text{id} \$$	error, skip $)$
$E \$$	$\text{id} * + \text{id} \$$	id is in $\text{FIRST}(E)$
$TE' \$$	$\text{id} * + \text{id} \$$	
$FT'E' \$$	$\text{id} * + \text{id} \$$	
$\text{id } T'E' \$$	$\text{id} * + \text{id} \$$	
$T'E' \$$	$* + \text{id} \$$	
$* FT'E' \$$	$* + \text{id} \$$	
$FT'E' \$$	$+ \text{id} \$$	error, $M[F, +] = \text{synch}$
$T'E' \$$	$+ \text{id} \$$	F has been popped
$E' \$$	$+ \text{id} \$$	
$+ TE' \$$	$+ \text{id} \$$	
$TE' \$$	$\text{id} \$$	
$FT'E' \$$	$\text{id} \$$	
$\text{id } T'E' \$$	$\text{id} \$$	
$T'E' \$$	$\$$	
$E' \$$	$\$$	
$\$$	$\$$	

Figure 4.23: Parsing and error recovery moves made by a predictive parser

Análisis Sintáctico Predictivo

- **Recuperación de errores. Recuperación a nivel de frases.**

Consiste en introducir punteros a rutinas de error en las celdas en blanco en la tabla de análisis.

Las rutinas pueden cambiar, eliminar o insertar caracteres a la entrada emitiendo los mensajes de error correspondientes.

Enfoque complicado de implementar